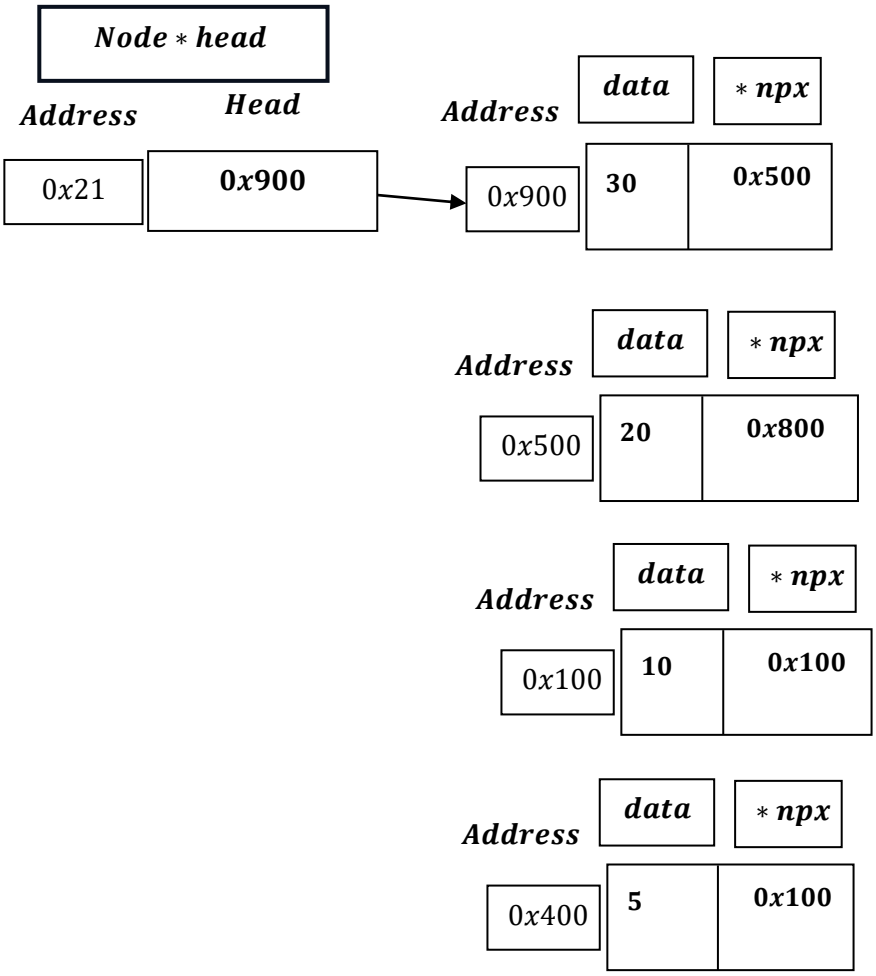
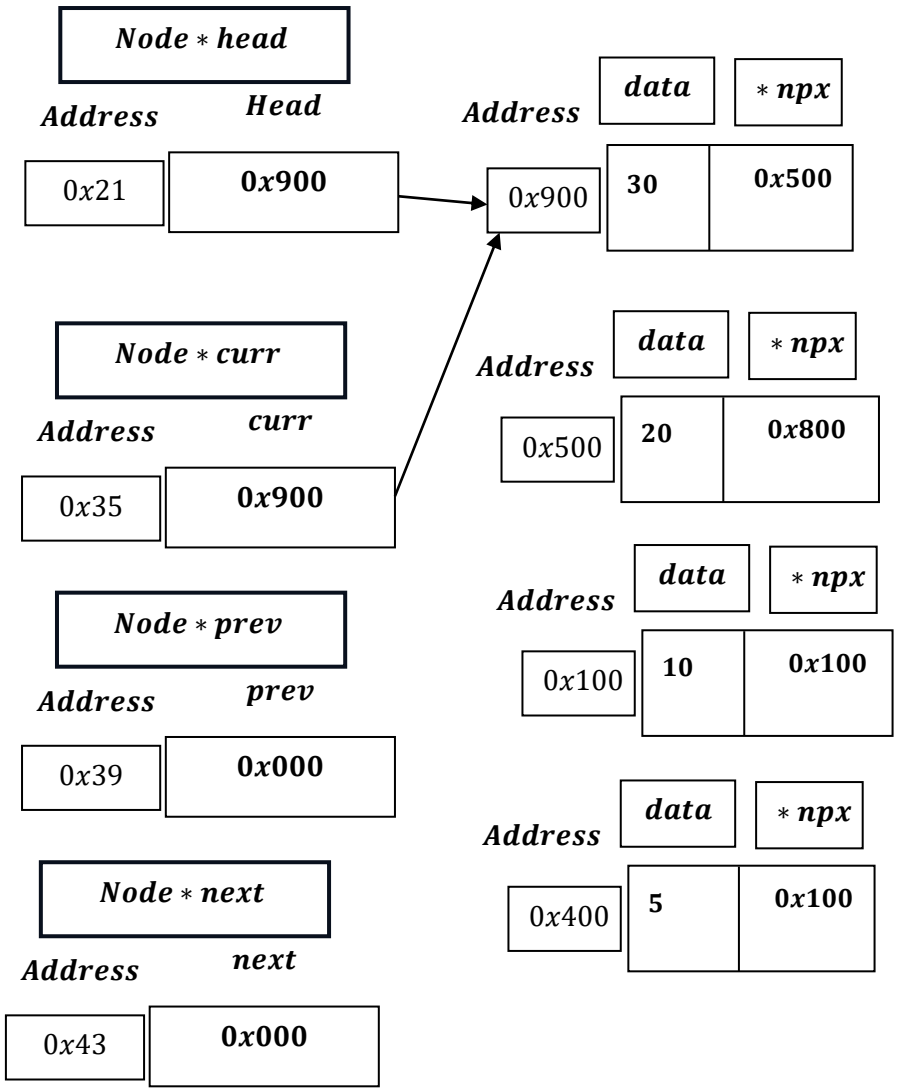
Memory Efficient Doubly Linked List – Delete At Position –Program Analysis – Part 3 and Time Complexity

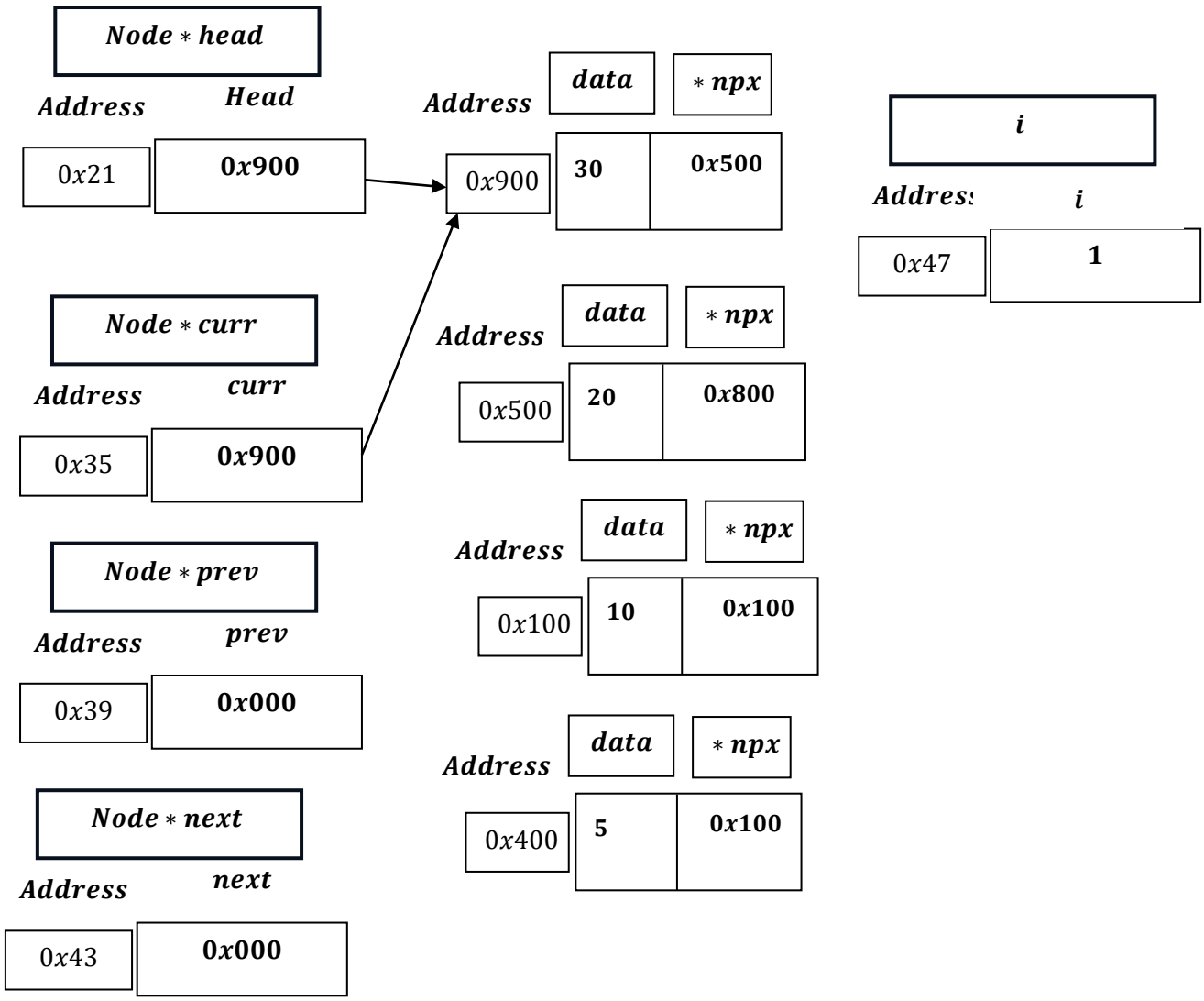
Position not Found.




```
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
```


Node * head
Address
Head
0x21
0x900
Node * curr
Address
curr
0x35
0x900
Node * prev
Address
prev
0x39
0x000
Node * next
Address
next
0x43
0x000
Address
data
* npx
0x900
30
0x500
Address
data
* npx
0x500
20
0x800
Address
data
* npx
0x100
10
0x100
Address
data
* npx
0x400
5
0x100

```
    int i = 1;
```




```
while (curr != nullptr && i < pos)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

***pos*: 5.**

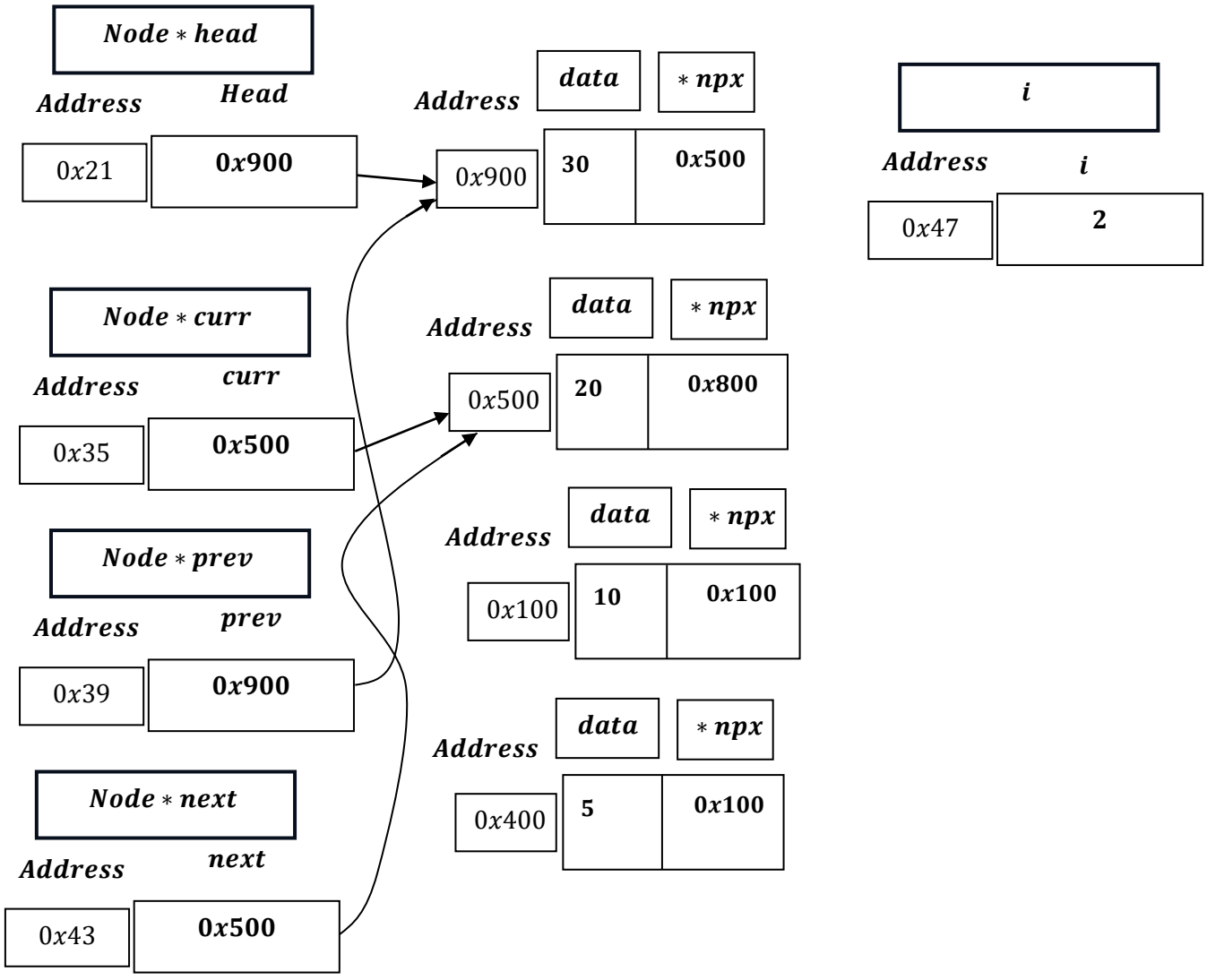
$\boldsymbol{curr: 0x900 \neq nullptr: 0x000\ AND\ i = 1 < pos: 5}$:

$\boldsymbol{next = XOR(prev: 0x000, curr \rightarrow npx: 0x500) = 0x000 \oplus 0x500 = 0x500.}$

$\boldsymbol{prev = curr = 0x900}$

$\boldsymbol{curr = next = 0x500}$

$\boldsymbol{i = i + 1 = 1 + 1 = 2.}$



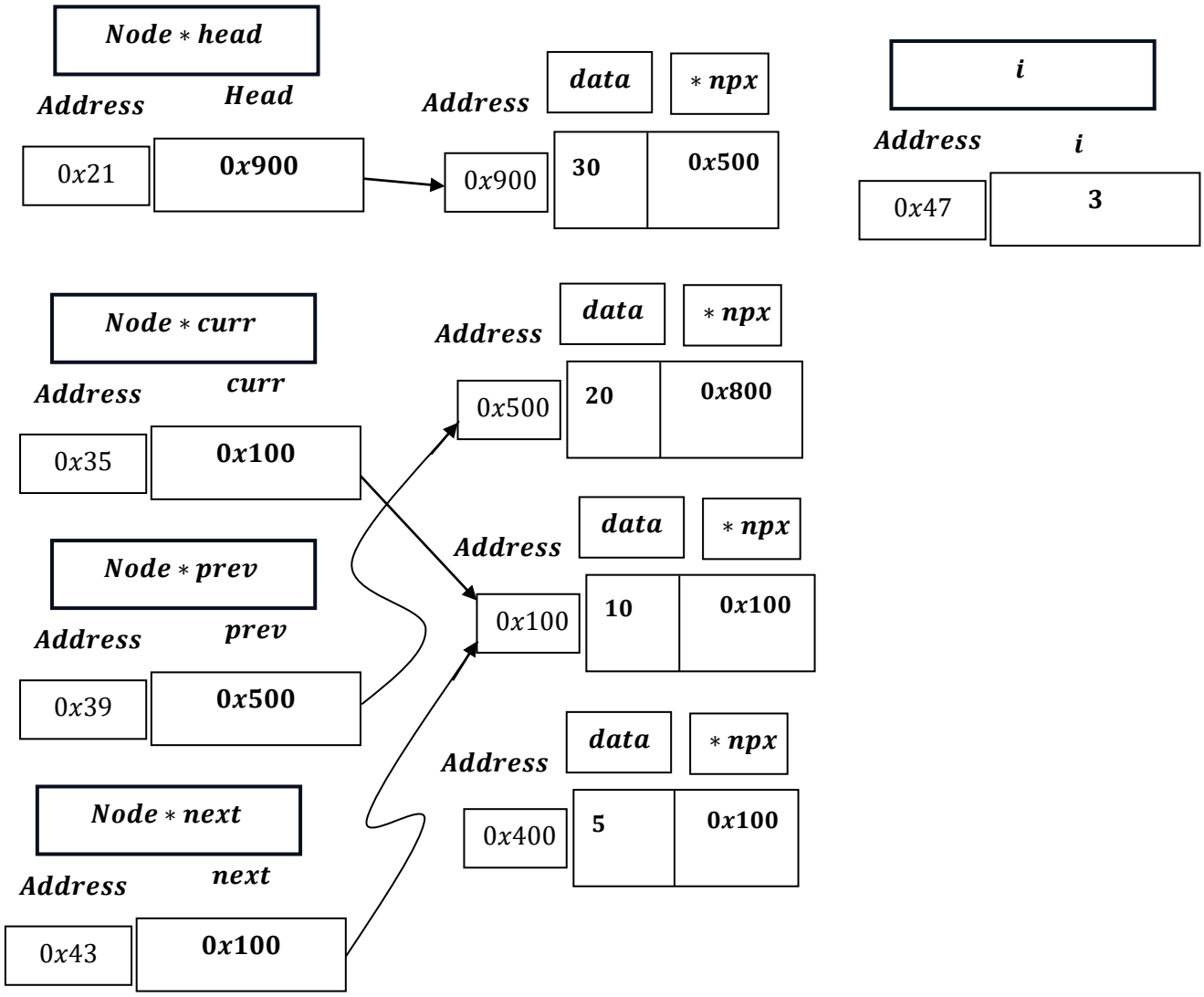
$curr: 0x500 \neq nullptr: 0x000\ AND\ i = 2 < pos: 5$:

$next = XOR(prev: 0x900, curr \rightarrow npx: 0x800) = 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500$

$curr = next = 0x100$

$i = i + 1 = 2 + 1 = 3$.



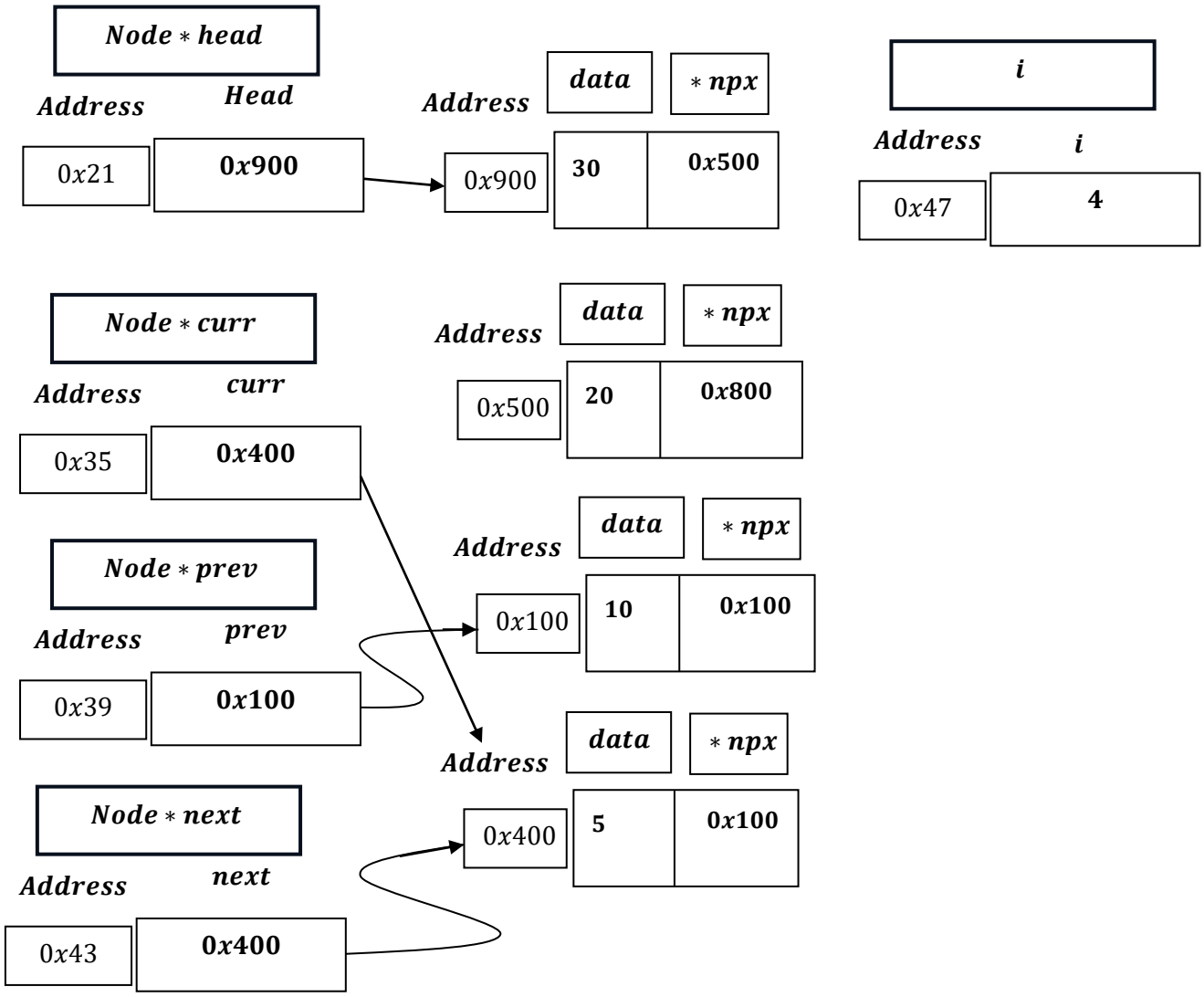
$curr: 0x100 \neq nullptr: 0x000\ AND\ i = 3 < pos: 5$:

$next = XOR(prev: 0x500, curr \rightarrow npx: 0x100) = 0101_2 \oplus 0001_2 = 0100_2 = 0x400$

$prev = curr = 0x100$

$curr = next = 0x400$

$i = i + 1 = 3 + 1 = 4$.



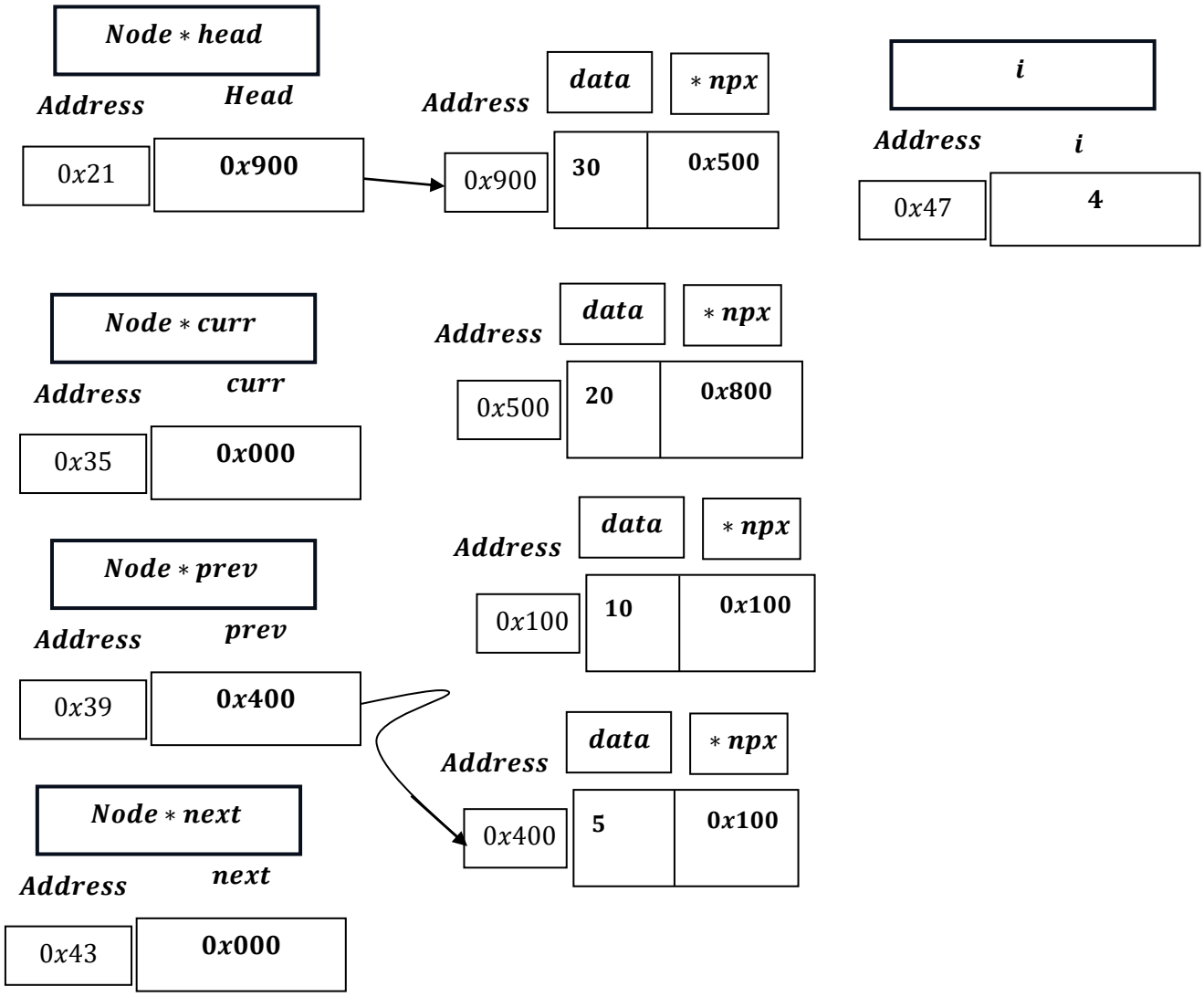
$\boldsymbol{curr: 0x400 \neq nullptr: 0x000\ AND\ i = 4 < pos: 5}$:

$\boldsymbol{next = XOR(prev: 0x100, curr \rightarrow npx: 0x100) = 0001_2 \oplus 0001_2 = 0000_2 = 0x000}$

$\boldsymbol{prev = curr = 0x400}$

$\boldsymbol{curr = next = 0x000}$

$\boldsymbol{i = i + 1 = 4 + 1 = 5}$.



$\boldsymbol{curr: 0x000 \neq nullptr: 0x000\ [False] AND\ i = 5 < pos: 5\ [False] \rightarrow Condition\ fails\ and\ loop\ exits}$.

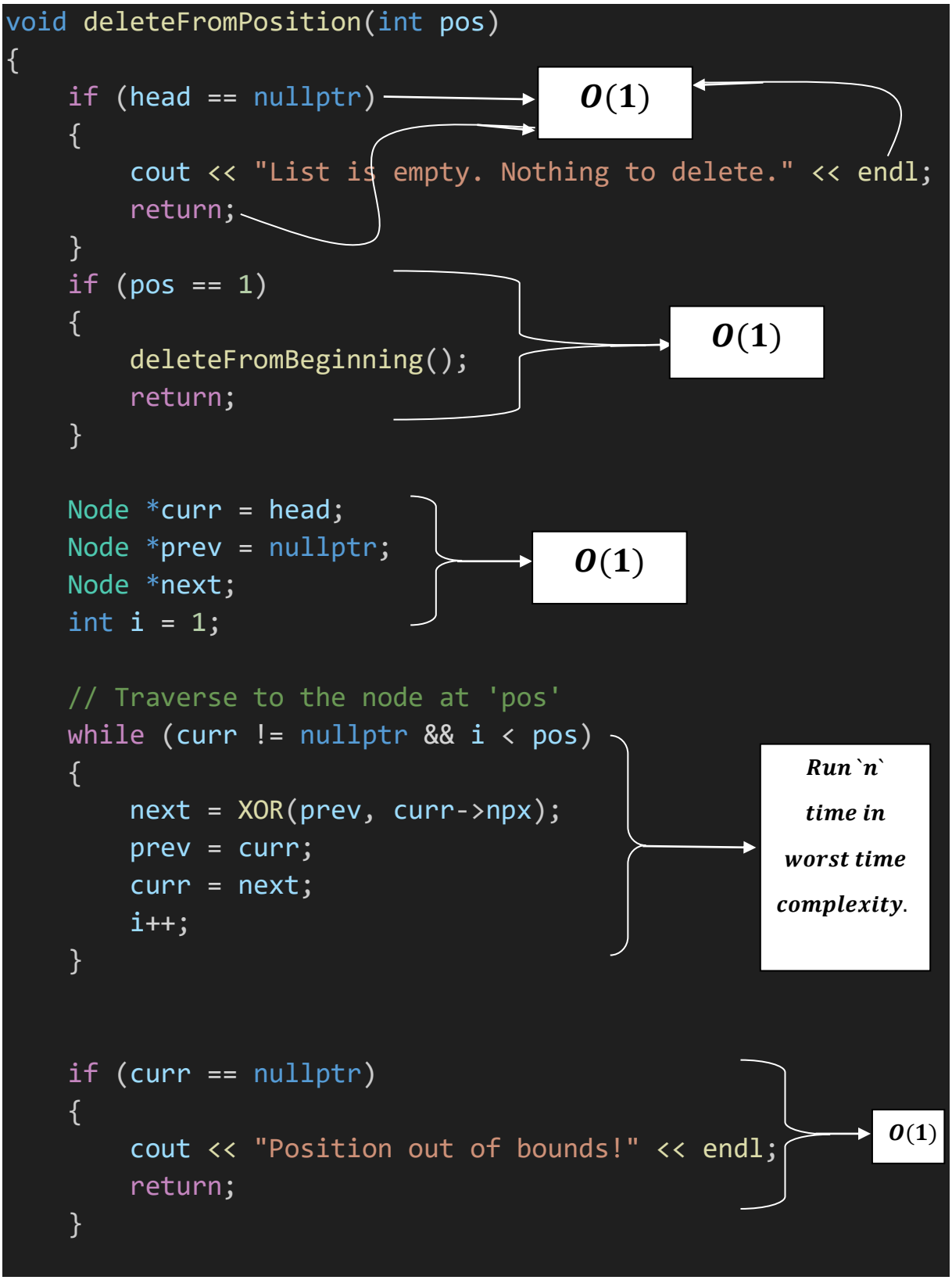
```
    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
```

$\boldsymbol{curr}: \boldsymbol{0x000} = \boldsymbol{nullptr}: \boldsymbol{0x000}:$

$\boldsymbol{Print}: ``\boldsymbol{Position\ out\ of\ bounds}!``$

$\boldsymbol{Return\ Void\ and\ exit\ from\ function}.$

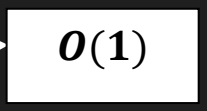
Time Complexity of Delete At Position

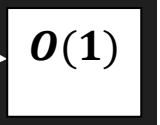


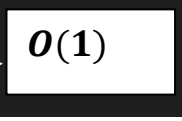
```
    next = XOR(prev, curr->npx);
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, next);

    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(prev, nextNext);
    }

    cout << "Deleted " << curr->data << " from position " <<
pos << "." << endl;
    free(curr);
}
```







Best Case(Absolute Best Case – When List is Empty)

List is Empty i.e. when head = nullptr, therefore , time complexity is : Ω(1).

Worst Time Complexity(Absolute Worst Case = `Pos` is not found).

while (curr != nullptr && i < pos) → runs n + 1 times when `pos` is not found:

next = XOR(prev, curr → npx); – runs `n` times.

prev = curr; – runs `n` times.

curr = next; – runs `n` times.

i = i + 1; – runs `n` times.

As we put forward the inner statements of the loop to calculate for worst case complexity :

$$\boldsymbol{4n = 4 \times O(n) = O(n).}$$

Average Case

Deleted position from 2 to N, while loop runs from 1 to N − 1.

And when the position is uniformly distributed from 1 to N:

(Average Case i.e. Undetermined Position)

→ ***If the position is 1, the loop's inner statement runs 0 time.***

→ ***If the position is 2, the loop's inner statement runs 1 time.***

→ ***If the position is 3, the loop's inner statement runs 2 time.***

.

.

→ ***If the position is (N), the loop's inner statement runs (N − 1) times.***

As when i = N < N becomes false , hence no execution of loop's inner statements.

$$Hence\ ,\ \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

Now, as we take all the possibilites then , we take the best case also i.e. O(1), when position = 1 . Considering `T(P)` is time complexity of position `P` .

$$T(P) = \begin{cases} 1 & ,when\ P = 1. \\ P - 1 & ,when\ P = 2, 3, 4, .., N. \end{cases}$$

$$Now, If\ there\ is\ `N`\ no.of\ nodes\ there\ we\ can\ delete\ data\ at\ \ N\ positions.$$

$$For\ position\ 1, the\ probability\ of\ deletion\ is \frac{1}{N}.$$

$$For\ position\ 2, the\ probability\ of\ deletion\ \ is \frac{1}{N}.$$

.

.

$$For\ position\ N, the\ probability\ of\ deletion\ \ is \frac{1}{N}.$$

$$Hence\ probability\ of\ deletion\ will\ always\ remain\ \frac{1}{N}.$$

$$\therefore\ Average\ Time\ Complexity\ \ is: \ probability\ of\ deletion \times (number\ of\ possible\ iterations + constant\ time\ complexities\ for\ left\ over\ positions)\ .$$

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \cdots + \frac{1}{N} \times N - 1\right) + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \cdots + N - 1) + 1\}$$

$$= \frac{1}{N} \times \left\{\sum_{i=1}^{N-1} i + 1\right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1\right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `$\frac{1}{2}$` ***we get***:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ **terms , largest growth rate is the result**:

$\rightarrow \Theta(N)$ ***grows linearly with*** `N` .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ ***decreases to a constant , i.e. when N grows larger ,*** $\frac{1}{N}$ ***approaches to*** 0.

This we can prove by limit and continuity also:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0 \text{ , where c is constant}\right].$$

As $\Theta(N)'s$ ***growth rate is higher , therefore , average case is*** $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

$$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$$

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

$$Average\ Case\ Time\ Complexity[Most\ Simplest\ Alternative\ Apporach]:$$

$$(Simplest\ approach)$$

$$For\ position\ 1, the\ probability\ of\ deletion\ is \frac{1}{N}.$$

$$For\ position\ 2, the\ probability\ of\ deletion\ \ is \frac{1}{N}.$$

.

.

$$For\ position\ N, the\ probability\ of\ deletion\ \ is \frac{1}{N}.$$

$$Hence\ probability\ of\ deletion\ will\ always\ remain\ \frac{1}{N}, for\ position\ 1\ to\ N.$$

$$Average\ case\ complexity: \ Probability\ of\ deletion \times \begin{pmatrix} Summation \\ of\ number\ of \\ Position. \end{pmatrix}$$

$$= \frac{1}{N} \times \left(\sum_{i=1}^{N} i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$or, \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant `$\frac{1}{2}$` ***we get***:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

$$\textit{Hence, Average Time Complexity is}: \ \Theta\left(\frac{N+1}{2}\right) \textit{ or } \Theta(N).$$

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, ***we get***:

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$

Memory Efficient Doubly Linked List	***Cases***	***Time Complexity***
Delete At Position	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case***	$\boldsymbol{\Theta(n)}$

****